

CS 3510 Algorithms: Bellman-Ford and Floyd-Warshall Algorithms

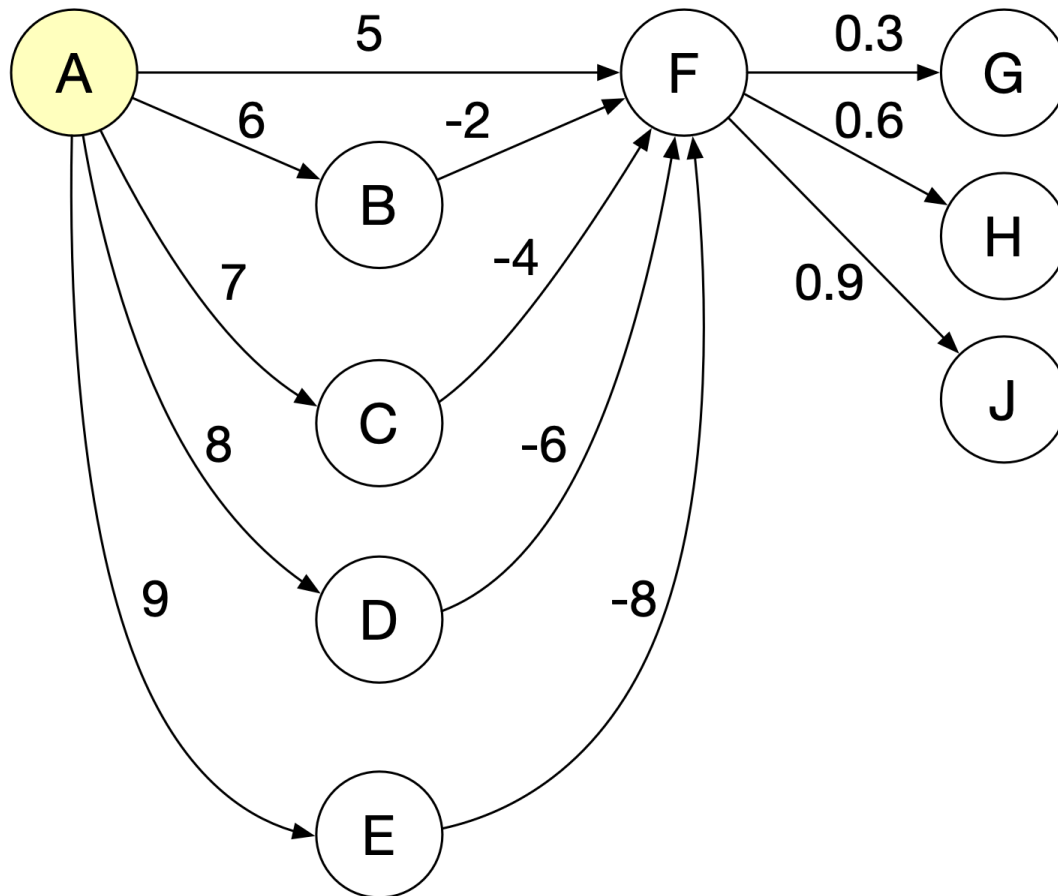
Aaron Hillegass
Georgia Tech

What makes Dijkstra's Algorithm great?

- Single source shortest path
- Each node is popped off the priority queue once: when it is the cheapest unpopped.
- Runtime: $O((|V| + |E|) \log(|V|))$

```
1  dist[v] := ∞ for all v ∈ G.V
2  dist[v₀] := 0.0
3  R := {v₀}
4  while R ≠ G.V:
5      Pick v ∉ R with lowest dist
6      forall (v, u), w leaving v:
7          if dist[v] + weight[v, u] < dist[u]:
8              dist[u] := dist[v] + weight[v, u]
9              prev[u] := v
10     Add v to R
```

Negative weights?

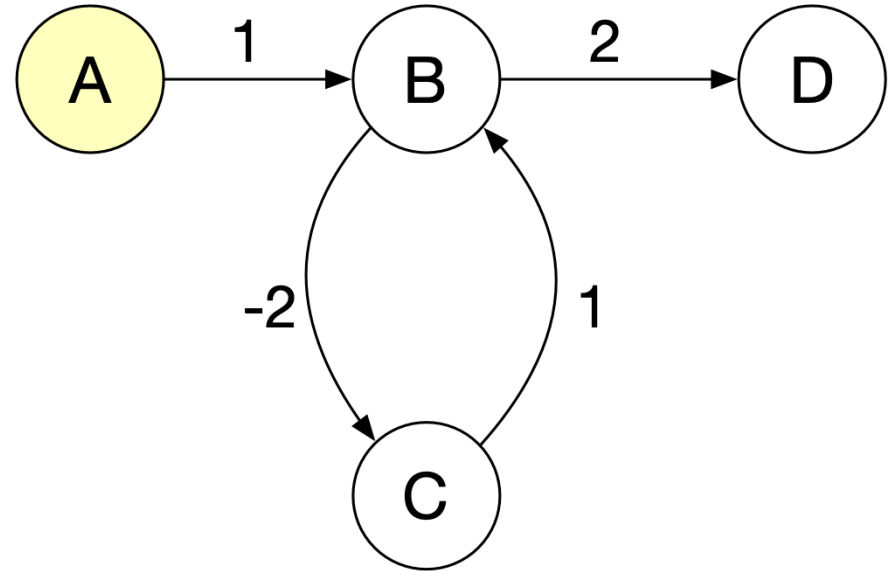


```

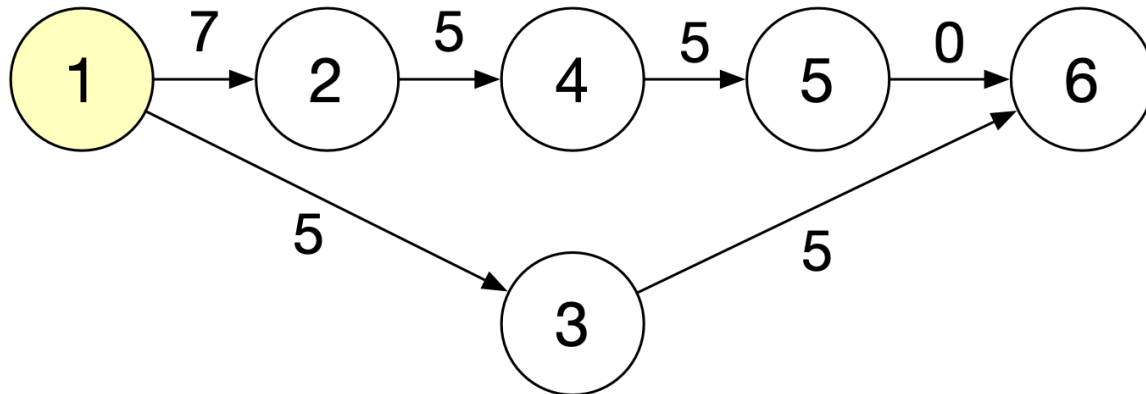
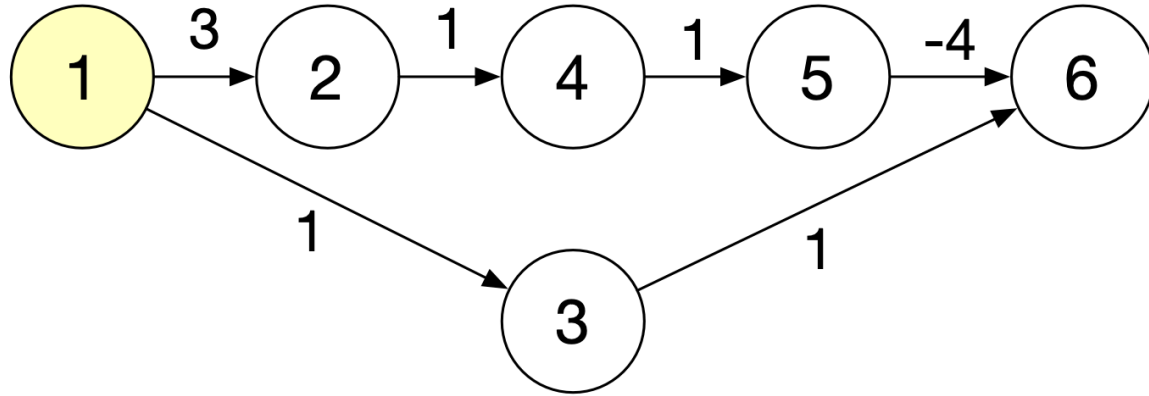
1  dist[v] := ∞ for all v ∈ G.V
2  dist[v₀] := 0.0
3  R := {v₀}
4  while R ≠ G.V:
5      Pick v ∉ R with lowest dist
6      forall (v, u), w leaving v:
7          if dist[v] + weight[V] < dist[u]:
8              dist[u] := dist[v] + weight[e]
9              prev[u] := v
10         Remove u from R
11  Add v to R
  
```

Negative cycles

```
1 dist[v] := ∞ for all v ∈ G.V
2 dist[v₀] := 0.0
3 R := {v₀}
4 while R ≠ G.V:
5     Pick v ∉ R with lowest dist
6     forall (v, u), w leaving v:
7         if dist[v] + weight[V] < dist[u]:
8             dist[u] := dist[v] + weight[e]
9             prev[u] := v
10        Remove u from R
11    Add v to R
```



Add a constant to all weights?

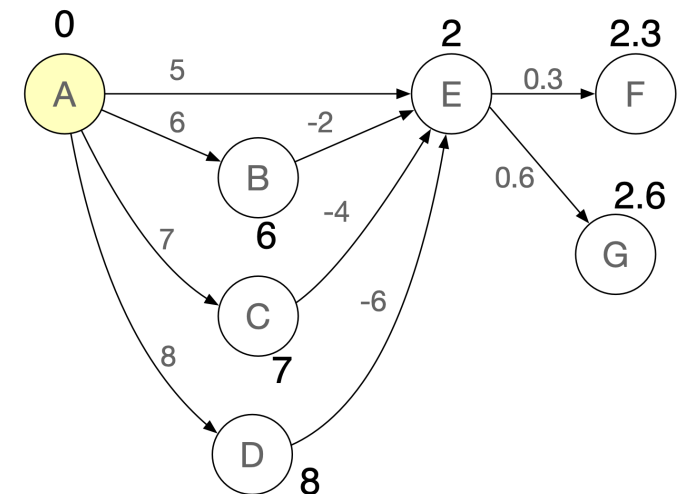
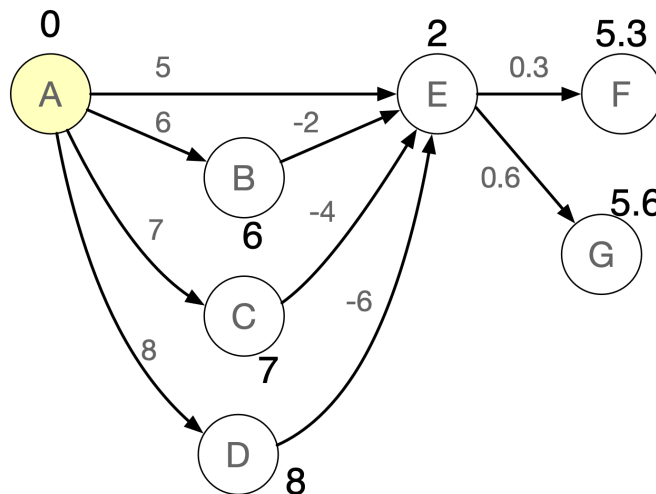
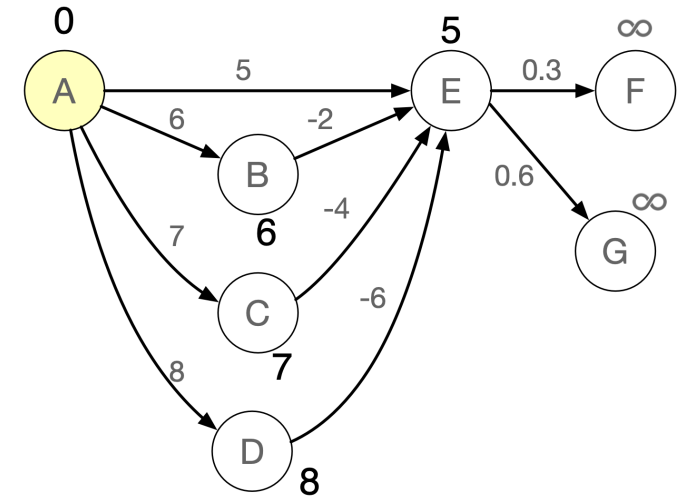
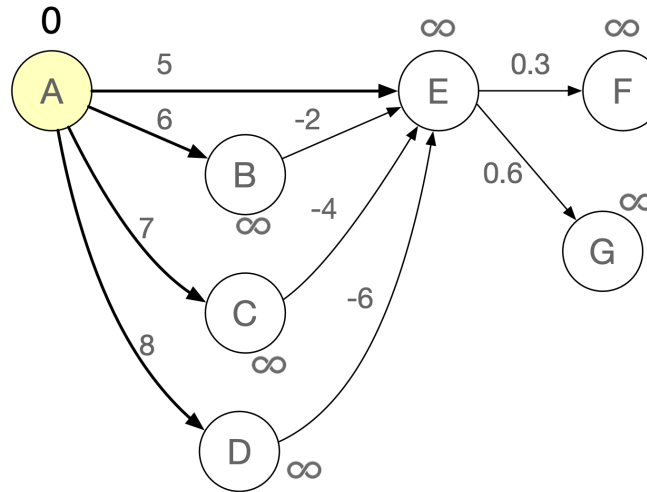


What if we just update repeatedly?

```

1  for  $v$  in  $G.V$ :
2     $\text{dist}[v] = \infty$ 
3     $\text{prev}[v] = \text{null}$ 
4   $\text{dist}[v_0] = 0.0$ 
5   $\text{is\_changed} := \text{True}$ 
6  while  $\text{is\_changed}$ :
7     $\text{is\_changed} := \text{False}$ 
8    for  $(u, v), w$  in  $G.E$ :
9      if  $\text{dist}[u] + w < \text{dist}[v]$ :
10          $\text{is\_changed} := \text{True}$ 
11          $\text{dist}[v] := \text{dist}[u] + w$ 
12          $\text{prev}[v] := u$ 
13  return  $\text{dist}, \text{prev}$ 

```



How many times?

The “truth” starts at v_0 and moves one edge per iteration.

If the graph has $|V|$ vertices, how long can the shortest path to any $u \in V$ be?

Answer: $|V| - 1$

If a graph has no negative cycles, dist stops changing after $|V| - 1$ iterations.

Bellman-Ford Algorithm

```
1 define BellmanFord( $G, v_0$ ):  
2   for  $v$  in  $G.V$ :  
3      $\text{dist}[v] = \infty$   
4      $\text{prev}[v] = \text{null}$   
5    $\text{dist}[v_0] = 0.0$   
6   loop  $|V| - 1$  times:  
7     for  $(u, v), w$  in  $G.E$ :  
8       if  $\text{dist}[u] + w < \text{dist}[v]$ :  
9          $\text{dist}[v] := \text{dist}[u] + w$   
10         $\text{prev}[v] := u$   
11  return  $\text{dist}, \text{prev}$ 
```


But what if there is a negative cycle?

Bellman-Ford Algorithm

10/21

```
1 define BellmanFord( $G, v_0$ ):
2   for  $v$  in  $G.V$ :
3      $\text{dist}[v] = \infty, \text{prev}[v] = \text{null}$ 
4    $\text{dist}[v_0] = 0.0$ 
5    $\text{count} := 0, \text{is\_changed} := \text{True}$ 
6   while  $\text{is\_changed}$  and  $\text{count} \neq |V|$ :
7      $\text{is\_changed} := \text{False}$ 
8     for  $(u, v), w$  in  $G.E$ :
9       if  $\text{dist}[u] + w < \text{dist}[v]$ :
10         $\text{is\_changed} := \text{True}$ 
11         $\text{dist}[v] := \text{dist}[u] + w, \text{prev}[v] := u$ 
12       $\text{count} := \text{count} + 1$ 
13   if  $\text{count} = |V|$  and  $\text{is\_changed}$ :
14     return failure (negative cycle)
15   return  $\text{dist}, \text{prev}$ 
```

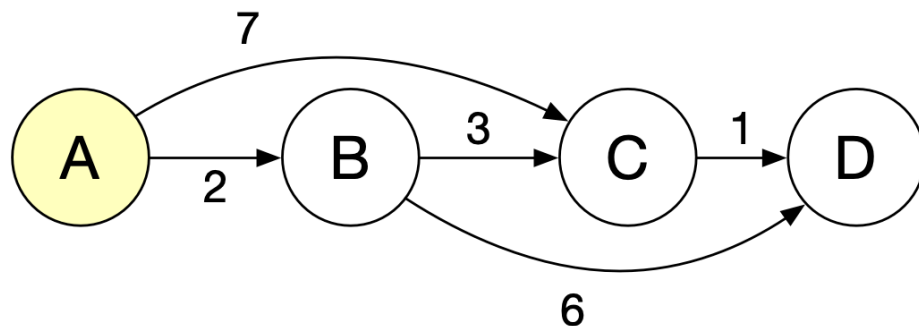
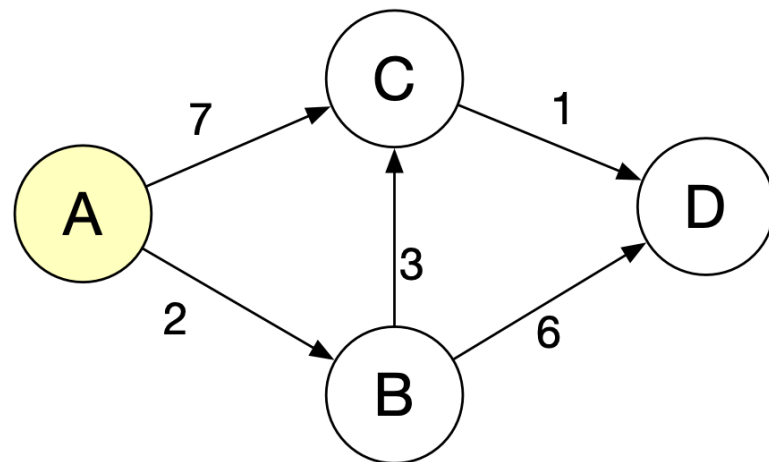
Worst case: $O(|V||E|) > O((|V| + |E|) \log(|V|))$

Bellman-Ford Algorithm

11/21

What if you know the the graph is a
DAG?

```
1 define SSSPDAD( $G, v_0$ ):  
2   for  $v$  in  $G.V$ :  
3      $\text{dist}[v] = \infty, \text{prev}[v] = \text{null}$   
4    $\text{dist}[v_0] = 0.0$   
5    $h := \text{topsort}(G)$   
6   for  $u$  in  $h$ :  
7     for  $(u, v), w \in G.E$ :  
8       if  $\text{dist}[u] + w < \text{dist}[v]$ :  
9          $\text{dist}[v] := \text{dist}[u] + w$   
10         $\text{prev}[v] := u$   
11  return  $\text{dist}, \text{prev}$ 
```



$$O(|V| + |E|)$$

What about shortest paths between all
pairs (u, v) ?

ASSP: Using what you know

Dijkstra's algorithm from every vertex:

$$O(|V|(|V| + |E|) \log |V|) \approx O(|V|^3 \log |V|)$$

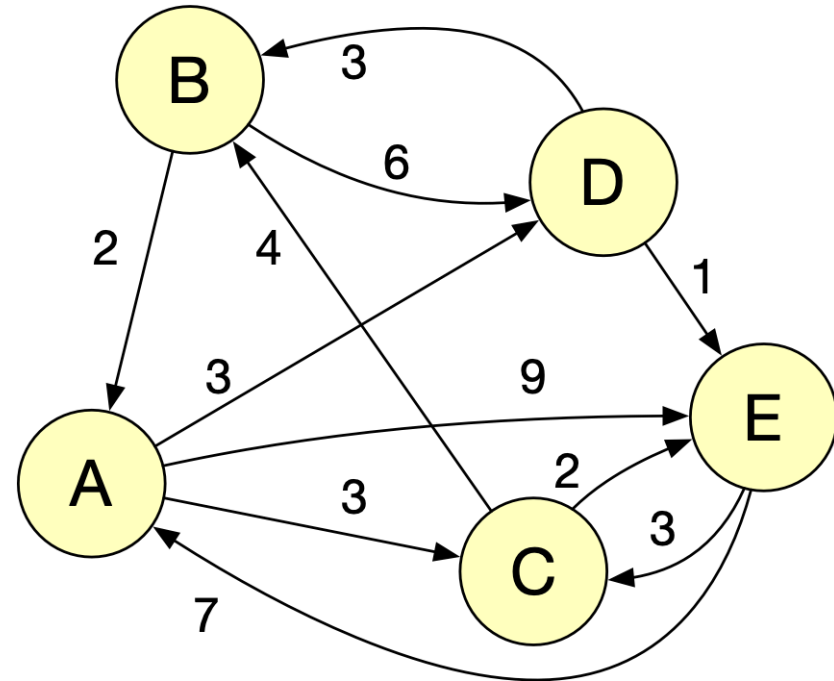
We can do a *little* better: $O(|V|^3)$

Result is a table

Fill in the given stuff.

(Not necessarily shortest!)

	To A	To B	To C	To D	To E
From A	0	∞	3	3	9
From B	2	0	∞	6	∞
From C	∞	4	0	∞	2
From D	∞	3	∞	0	1
From E	7	∞	3	∞	0



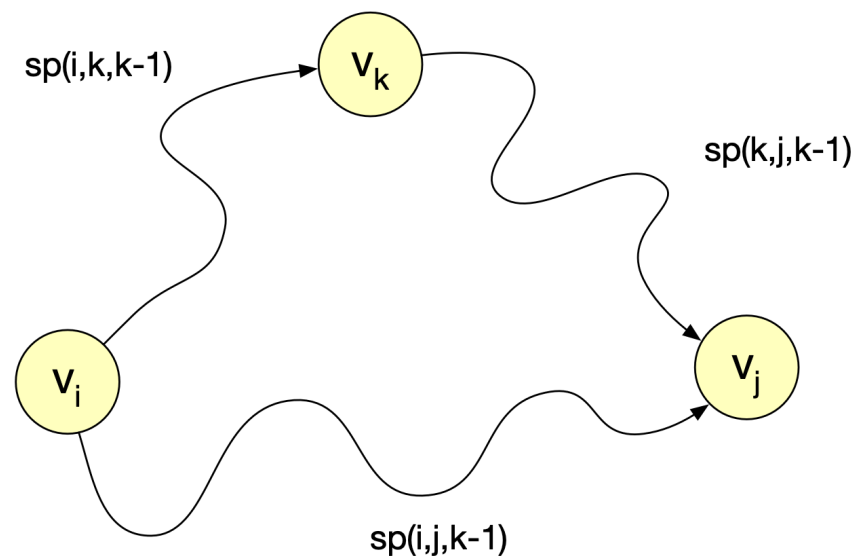
Let the vertices be numbered: $[v_1, v_2, \dots, v_n]$

Let $\text{shortestPath}(i, j, k)$ be the shortest path from v_i to v_j by only passing through vertices from the set $\{v_1, v_2, \dots, v_k\}$.

When looking for $\text{shortestPath}(i, j, k)$ there are only two possibilities:

- It contains v_k : It is $\text{shortestPath}(i, k, k-1) + \text{shortestPath}(k, j, k-1)$
- It does not contain v_k : It is $\text{shortestPath}(i, j, k-1)$

Figure out both, and take the minimum.



Step 1: Fill in the base cases

```

1 forall  $i, j$ :
2   |  $\text{dist}[i, j] := \infty$ 
3 for  $i \in 1..|V|$ :
4   |  $\text{dist}[i, i] := 0$ 
5 for  $(v_i, v_j), w$  in  $E$ :
6   |  $\text{dist}[i, j] := w$ 

```

	To A	To B	To C	To D	To E
From A	0	∞	3	3	9
From B	2	0	∞	6	∞
From C	∞	4	0	∞	2
From D	∞	3	∞	0	1
From E	7	∞	3	∞	0

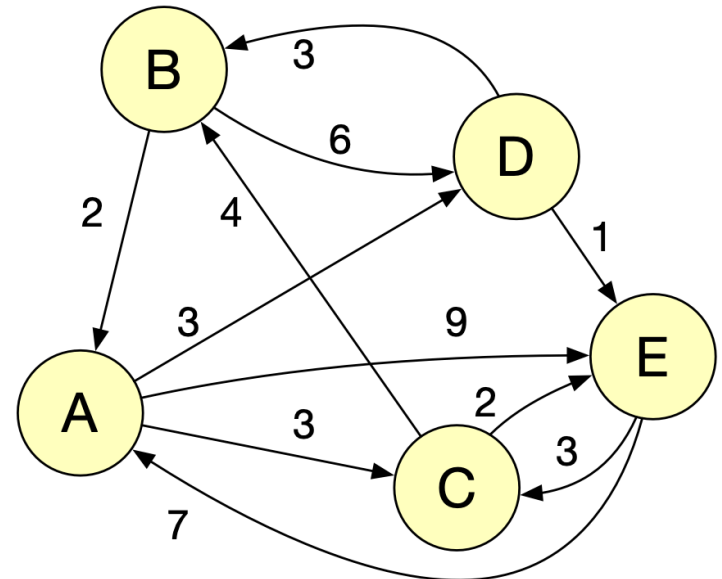
Step 2: Loop!

```

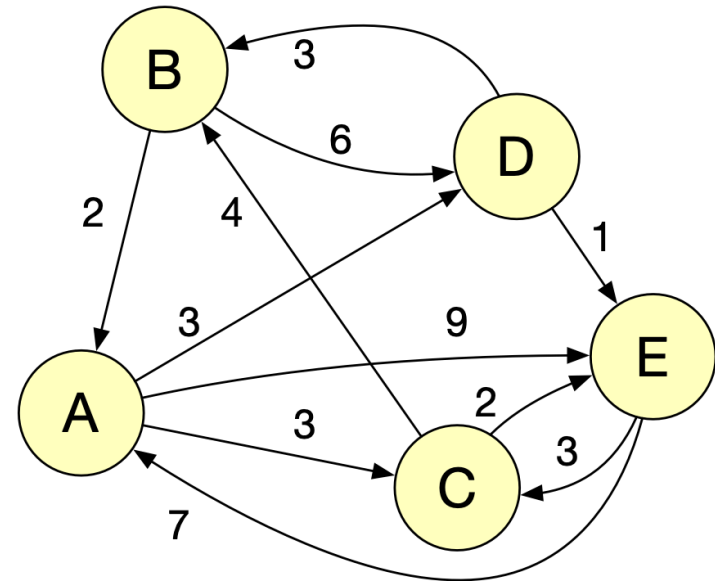
1 for  $k \in 1..|V|$ :
2   for  $i \in 1..|V|$ :
3     for  $j \in 1..|V|$ :
4        $\text{dist}[i, j] := \min(\text{dist}[i, j], \text{dist}[i, k] + \text{dist}[k, j])$ 

```

	To A	To B	To C	To D	To E
From A	0	∞	3	3	9
From B	2	0	∞	6	∞
From C	∞	4	0	∞	2
From D	∞	3	∞	0	1
From E	7	∞	3	∞	0



	To A	To B	To C	To D	To E
From A	0	6	3	3	5
From B	2	0	5	6	7
From C	9	4	0	10	2
From D	8	3	4	0	1
From E	7	7	3	10	0



Questions?

Slides by Aaron Hillegass